
Exame de Estruturas de Dados – 2009.Junho.27

Duração: 2:30 horas

Valorização: 15 ou 16 valores.

1. Responda às seguintes questões:

- a) Coloque por ordem crescente de complexidade: $O(50 \log N)$, $O(300N^2)$, $O(N)$, $O(5^3)$, $O(2N \log N)$.
- b) Faça o estudo da complexidade do seguinte algoritmo:

```
for (i=0; i<n; i++)
    for (j=0; j<i*i; j++)
        A[j % n] = j;
```

- c) Suponha que tem uma classe **Campeonato** que tem pelo menos dois atributos, um inteiro **ne** que representa o número de equipas e uma matriz **r**[],[], com $ne \times ne$ valores, que contém os resultados dos jogos entre as equipas num campeonato a uma volta, todos contra todos. O elemento **r**[i][j] representa os golos marcados pela equipa **i** à equipa **j** no jogo entre as duas equipas. Assim, se por exemplo, **r**[2][3]=5 e **r**[3][2]=2, isso significa que a equipa 2 ganhou à equipa 3 por 5 a 2.

Escreva um método que, a partir da matriz **r**[],[], calcule um vector **p**[], com a pontuação de cada equipa devolvendo esse vector como resultado. A pontuação de um jogo é de 3 pontos para a equipa vencedora, 1 ponto para cada equipa em caso de empate e 0 pontos para a equipa vencida.

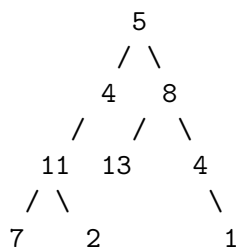
2. Listas

Numa corrida de carros os concorrentes têm um período de tempo em que são livres de fazer as voltas que quiserem à pista, no sentido de tentarem conseguir o tempo mais rápido possível. Estes tempos conseguidos pelos diferentes concorrentes são registados para no final do período se poder saber qual o melhor tempo (o mais baixo) para cada concorrente. Cada tentativa é registada com o número do concorrente (um inteiro) e o número de segundos que levou a fazer a volta à pista (pode assumir também que se trata de um inteiro).

- a) Implemente a estrutura de dados para armazenar esta informação usando listas ligadas. Desenvolva as estruturas de dados necessárias (classes), escreva o método construtor de objectos, um método para inserir um novo registo temporal de um concorrente e um método que escreva os tempos de um determinado concorrente identificado pelo seu número.
- b) Considere que existem no máximo 20 concorrentes e que os números de concorrente são de 100 a 119. Implemente um método que obtenha os melhores tempos (tempos menores) de cada concorrente. Tais tempos devem ser devolvidos na estrutura que considere mais adequada para o efeito.
- c) Por vezes os concorrentes cometem infracções graves que levam os organizadores a desqualificá-los. Implemente um método que remova os tempos todos de um determinado concorrente (identificado pelo seu número), em função de ter sido desqualificado.

3. Árvores binárias

Definimos um caminho Raiz-folha numa árvore binária de números inteiros como uma sequência de nós que começa na raiz e prossegue pela árvore até chegar a uma folha (um nó sem descendentes). A árvore vazia não possui caminhos Raiz-folha. Por exemplo a árvore seguinte possui exactamente quatro caminhos Raiz-folha:



Caminhos Raiz-folha:

caminho 1: 5 4 11 7

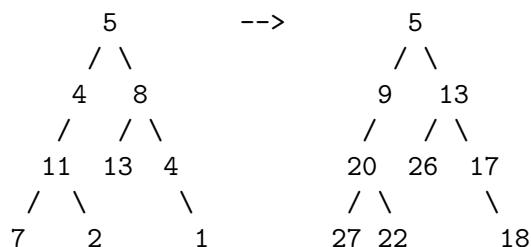
caminho 2: 5 4 11 2

caminho 3: 5 8 13

caminho 4: 5 8 4 1

Para esta questão estamos interessados em considerar a soma dos valores inteiros dos nós que constituem um caminho Raiz-folha. Por exemplo o valor soma do caminho 5-4-11-7 é 27.

- Escreva uma classe java **Node**, com pelo menos um construtor adequado, para um nó de uma árvore binária de números inteiros. Escreva uma classe java **Tree** para uma árvore binária, não se esquecendo também de incluir um construtor apropriado.
- Escreva um método void **substituiPorSoma()** para a classe **Tree** que substitui todos os nós da árvore pelo respectivo valor da soma dos nós compreendidos entre esse nó e a raiz da árvore. Exemplo:



- Escreva um método boolean **temCaminhoSoma(int soma)** que aceita um valor inteiro **soma** como argumento e retorna **true** se a instância da árvore possui pelo menos um caminho Raiz-folha com um valor soma igual àquele que é fornecido como argumento e retorna **false** se um tal caminho Raiz-folha não existir na instância da árvore onde o método é invocado. (Obs: se vir que o método anterior ajuda, pode usá-lo).